RAG & Context Engineering

Core Concepts, Design Decisions, and Failure Modes

@techNmak

Questions in this handbook

1. What is retrieval-augmented generation (RAG), and what does it add to a language model that the model's parameters alone do not provide?
2. How is context engineering broader than prompt engineering, and where does RAG fit inside it?
3. What are the major stages of a production RAG pipeline, and why should parsing, retrieval, context assembly, and generation be evaluated separately?
4. When should you use RAG instead of long-context prompting, fine-tuning, or continued pretraining?
5. Why can document parsing fail before retrieval even begins, especially for PDFs, tables, and structured documents?
6. Why is there no universally correct chunk size for RAG? What changes as chunks become smaller or larger?
7. What does chunk overlap actually solve, and how can too much overlap hurt retrieval and generation?
8. How should you choose between fixed-size, structure-aware, and semantic chunking?
9. How do Contextual Retrieval and Late Chunking try to preserve document-level context, and how are they different?
10. What is hierarchical retrieval, and why can RAPTOR-like indexes help with long documents or questions that require multiple levels of abstraction?
11. How should you choose top-k, and why can retrieving more passages make the final answer worse?
12. Why can the order and position of retrieved evidence inside the context window change model performance?
13. What does reranking contribute after retrieval, and why is it still useful even when the first-stage retriever is strong?
14. Why rewrite or expand a user's query before retrieval, and how can query transformation accidentally change the user's intent?
15. What is HyDE, and why must a hypothetical document generated by the model never be treated as evidence?
16. When can multi-query retrieval and rank fusion help, and why is generating more query variants not automatically better?
17. Why can one-shot retrieval fail on multi-hop questions, and what does interleaving retrieval with reasoning change?
18. Should every request retrieve a fixed number of passages? How do active, adaptive, and corrective RAG approaches change that assumption?
19. What is context compression or selective augmentation, and when can compressing retrieved evidence make the system less reliable?
20. How should a RAG system handle redundant, irrelevant, and mutually conflicting retrieved evidence?
21. What happens when retrieved evidence conflicts with the model's parametric knowledge? Which source will the model follow?
22. Why does attaching a citation to an answer not prove that the cited source supports the claim or that the model actually relied on it?
23. Why can a RAG system hallucinate even when the retriever returned the correct evidence?
24. How should a production RAG system handle freshness, provenance, updates, deletions, and re-indexing?
25. How should permissions be enforced in RAG so that unauthorized documents never become model context?
26. How can poisoned or instruction-bearing documents attack a RAG system, and why should retrieved text be treated as untrusted data?
27. Which metrics should you use to evaluate retrieval, context quality, generation, attribution, and the end-to-end RAG system?
28. What do RAGAS, ARES, and RAGChecker actually evaluate, and why are automated RAG judges not ground truth?
29. What problem is GraphRAG designed to solve, and when is a graph-based index unnecessary or worse than ordinary RAG?
30. Design a production RAG system for a changing enterprise corpus with permissions, citations, multi-hop questions, long documents, and a strict latency SLO. How would you prove each layer works?

About this handbook

This handbook focuses on retrieval-augmented generation and the engineering of the context supplied to language models. It assumes the mechanics of embeddings, lexical/dense retrieval, ANN indexes, and reranking are covered separately. The goal here is the layer above retrieval: ingestion, chunking, query transformation, evidence selection, context packing, attribution, security, evaluation, and production architecture. Results from individual papers are stated as results of those studies rather than universal laws.

Questions and answers

1. What is retrieval-augmented generation (RAG), and what does it add to a language model that the model's parameters alone do not provide? [Beginner]

RAG augments generation with information retrieved from an external knowledge source at inference time. The model still contributes its parametric knowledge and language-generation ability, but the prompt or model computation is conditioned on retrieved non-parametric evidence. In the original RAG formulation, the retriever assigns probabilities to documents and the generator conditions on both the input and retrieved document; the central idea is that useful knowledge need not be stored only in model weights. This makes external knowledge easier to update and gives the system a path to provenance. It does not make answers automatically factual: the retriever can miss the evidence, the corpus can be wrong, and the generator can ignore or misuse correct context. RAG is therefore best understood as an evidence-access architecture, not a hallucination switch. [1]

2. How is context engineering broader than prompt engineering, and where does RAG fit inside it? [Intermediate]

Prompt engineering is mainly about writing and organizing instructions. Context engineering is broader: it is the design of the complete token state presented to the model at inference time - instructions, retrieved evidence, tool results, conversation state, metadata, examples, and other information that may affect behavior. RAG is one mechanism for acquiring part of that context from an external corpus. A good RAG retriever can still produce a poor system if the retrieved passages are badly ordered, redundant, overlong, stale, unauthorized, or mixed with ambiguous instructions. Conversely, context engineering can exist without vector retrieval at all; a system may load files just in time, use structured queries, cached context, or tool outputs. The practical goal is not to maximize context length. It is to supply a small enough, high-signal context that is sufficient for the task and safe for the model to use. [2]

3. What are the major stages of a production RAG pipeline, and why should parsing, retrieval, context assembly, and generation be evaluated separately? [Intermediate - System Design]

A production pipeline commonly has at least these stages: source ingestion and parsing; document segmentation; indexing; query interpretation; candidate retrieval; optional fusion or reranking; authorization filtering; context selection and ordering; generation; attribution; and logging/evaluation. Each stage creates different failures. A table may be parsed incorrectly even if retrieval is perfect. The right passage may be absent from the candidate set. A reranker may discard it. The passage may be present in the prompt but the generator may ignore it. An answer may be correct yet cite the wrong source. End-to-end accuracy alone hides these distinctions. The UDA benchmark, for example, explicitly studies parsing, retrieval, generation, and end-to-end document analysis as separate design choices. The engineering lesson is to retain enough intermediate artifacts - parsed units, retrieved IDs and scores, reranked context, final prompt, citations - to test the pipeline layer by layer. [3]

4. When should you use RAG instead of long-context prompting, fine-tuning, or continued pretraining? [Advanced - Judgment]

Use RAG when the task depends on a corpus that is too large, too dynamic, too permission-sensitive, or too provenance-critical to place entirely in the prompt or model weights. Long-context prompting can be simpler when the relevant corpus is small enough to fit economically and the model can reliably use it. Fine-tuning is better suited to changing behavior, style, task competence, or learned patterns; it is a poor substitute for frequently changing factual records that must be updated or cited. Continued pretraining can adapt a model to a domain, but it also changes weights and does not give document-level provenance. There is no universal RAG-versus-long-context winner. A 2024 study found long context stronger on average under some sufficiently resourced settings, while the 2025 LaRA benchmark found the best choice depends on model, context length, task type, and retrieval characteristics. Route based on measured quality, cost, latency, freshness, and governance requirements. [26][27]

5. Why can document parsing fail before retrieval even begins, especially for PDFs, tables, and structured documents? [Intermediate - Production]

Retrieval can only search what ingestion preserved. Real PDFs and web documents contain headings, columns, footnotes, tables, page furniture, reading-order ambiguities, scanned text, and sometimes figures whose structure disappears under naive text extraction. A parser can therefore corrupt the evidence before chunking or embedding

happens. For example, a table converted into a flat stream may separate a value from its row and column labels; retrieval may later return the correct-looking number with the wrong meaning. UDA was designed around raw PDFs/HTML and specifically highlights parsing as an end-to-end RAG bottleneck. Production systems should evaluate extraction on representative document types, preserve source coordinates and structural metadata where useful, and treat tables or layout-heavy material with parsers or multimodal methods appropriate to the source. Better embeddings cannot recover information that parsing destroyed. [3]

6. Why is there no universally correct chunk size for RAG? What changes as chunks become smaller or larger? [Intermediate - Trade-off]

Chunk size controls a real trade-off. Very small chunks can localize a fact precisely and reduce irrelevant tokens, but they can sever definitions, qualifiers, headers, or multi-sentence reasoning from the evidence they explain. Large chunks preserve more local context, but their embeddings can blur multiple topics, retrieval can become less selective, and the generator receives more irrelevant tokens. The best size therefore depends on document structure, embedding model, query granularity, retrieval strategy, and the kind of answer expected. Recent document-segmentation work likewise finds that segmentation materially affects retrieval and QA, but does not establish one universal token count. A defensible workflow is to generate or label representative queries, sweep several chunk sizes/strategies, measure retrieval evidence coverage and end-to-end answer quality, and inspect boundary failures. A copied default such as '512 tokens' is a starting configuration, not an engineering conclusion. [4]

7. What does chunk overlap actually solve, and how can too much overlap hurt retrieval and generation? [Intermediate]

Overlap is mainly a boundary hedge: if an important span crosses a chunk boundary, repeating some neighboring text can allow at least one chunk to contain enough of the evidence. The cost is duplication. More overlap creates more vectors or index entries, increases ingestion and storage, and can make several nearly identical chunks occupy the top-k results. That wastes context budget and can overweight repeated evidence during generation. It can also make evaluation look better at the chunk level because the same fact appears multiple times without increasing independent evidence. There is no universal percentage that is always optimal. Use overlap when your segmentation method creates harmful boundary cuts, then measure whether it actually improves evidence recall. Structure-aware segmentation, parent-child retrieval, contextualized chunks, or retrieving neighboring sections after the initial hit can sometimes preserve continuity with less duplication. The objective is sufficient evidence, not maximum token reuse.

8. How should you choose between fixed-size, structure-aware, and semantic chunking? [Advanced - Judgment]

Fixed-size chunking is cheap, deterministic, and a useful baseline. Structure-aware chunking follows boundaries such as sections, paragraphs, code blocks, or table units and is often preferable when the source format contains meaningful hierarchy. Semantic chunking tries to place boundaries where topics change, but adds model or embedding dependence and is not guaranteed to outperform simpler methods on every corpus. Findings ACL 2025 work on document segmentation shows that overly large and overly small segments create different failure modes and that segmentation can improve both retrieval and end-to-end QA, while a 2026 evaluation on academic texts also found a tested cluster-based semantic chunking method did not outperform simpler fixed-size or recursive strategies. Choose by source structure and measured evidence retrieval. Preserve stable document IDs and parent hierarchy so the system can retrieve a narrow child for precision while still expanding to its section or document when broader context is needed. [4][31]

9. How do Contextual Retrieval and Late Chunking try to preserve document-level context, and how are they different? [Advanced]

Both techniques address the same failure: a chunk may be locally understandable to a human who saw the whole document, yet ambiguous when embedded alone. Anthropic's Contextual Retrieval generates a short chunk-specific description from the full document and prepends that context before building semantic and lexical indexes. Late Chunking takes a different route: a long-context embedding model first computes token representations while the tokens can attend to the larger document, and only then are token representations pooled into chunk vectors. The former changes the text being indexed and can work with ordinary downstream embedding/BM25 pipelines; the latter changes where chunking occurs inside the embedding computation and requires a model that supports the method. Neither is a universal replacement for normal chunking. Both add cost or model assumptions and should be evaluated on the actual corpus. Anthropic's reported gains are results from its own experiments, not a general guarantee. [5][6]

10. What is hierarchical retrieval, and why can RAPTOR-like indexes help with long documents or questions that require multiple levels of abstraction? [Advanced]

Flat chunk retrieval represents a long document only at one granularity. Hierarchical retrieval adds multiple levels - for example, passages, sections, summaries, or clusters - so a query can retrieve either specific evidence or broader abstractions. RAPTOR recursively embeds, clusters, and summarizes text to build a tree whose higher nodes

represent progressively broader information. At query time it can retrieve across levels, which was designed to help questions that require holistic or multi-step understanding rather than one local span. The trade-off is extra indexing cost and possible summary error: an abstractive parent node is itself model-generated and can omit or distort detail. A common production pattern is therefore to use higher-level nodes to navigate or broaden recall but retain links to original source passages for evidence and citation. Hierarchy is valuable when document structure or question scope demands it; it is unnecessary complexity for small corpora dominated by local fact lookup. [7]

11. How should you choose top-k, and why can retrieving more passages make the final answer worse? [Intermediate - Context]

Top-k is a context-budget decision, not a retrieval constant. Raising k may improve the chance that supporting evidence is present, especially when the retriever's ranking is imperfect. But every additional passage can add irrelevant, redundant, conflicting, or lower-quality text. That increases input cost and may make evidence harder for the generator to identify. Self-RAG specifically motivates adaptive retrieval partly because blindly retrieving a fixed number of passages can be unhelpful. The correct k depends on candidate recall curves, reranking quality, average passage size, model context behavior, and answer type. Evaluate retrieval recall at several depths and then evaluate generation with the actual packed context. Often the right design is to retrieve more candidates cheaply, rerank them, deduplicate them, and pass a smaller evidence set to the model rather than simply increasing prompt k. [14]

12. Why can the order and position of retrieved evidence inside the context window change model performance? [Intermediate - Context]

Long-context models do not necessarily use all positions equally well. Lost in the Middle showed substantial position sensitivity on multi-document QA and key-value retrieval: models in that study were often strongest when relevant information appeared near the beginning or end and weaker when it appeared in the middle. The exact pattern is model- and task-dependent, so it should not be turned into a universal rule such as 'always put the best chunk last.' The durable lesson is that context packing is part of the model interface. Rank the most important evidence deliberately, group related passages, avoid unnecessary filler, and test permutations on representative tasks. Reordering can also interact with citations, chronological reasoning, and contradictory sources. A model advertising a large context window tells you how many tokens it can accept, not that every token position will contribute equally to answer quality. [8]

13. What does reranking contribute after retrieval, and why is it still useful even when the first-stage retriever is strong? [Intermediate - System Design]

First-stage retrieval is optimized to search a large corpus efficiently, so it often uses cheaper lexical or embedding scores. Reranking spends more computation on a much smaller candidate set using a stronger relevance model or richer query-document interaction. In RAG, its value is not merely a higher search metric: it determines which evidence consumes the limited generator context. A better reranker can remove plausible-but-wrong chunks and move truly supporting passages into the prompt. The trade-off is extra latency and cost, and reranking only helps documents that the first stage retrieved. A useful design is therefore to measure first-stage evidence recall at candidate depth, reranker gains at several cutoffs, and end-to-end answer quality. Anthropic's Contextual Retrieval experiments, for example, reported additional retrieval improvements when reranking was added, but those numbers are specific to its evaluated corpora and models. [5]

14. Why rewrite or expand a user's query before retrieval, and how can query transformation accidentally change the user's intent? [Advanced]

User queries are often underspecified, short, or expressed with vocabulary that differs from the corpus. Query expansion or rewriting can add synonyms, disambiguating terms, likely entities, or a more document-like representation. Query2doc, for example, generates pseudo-documents and uses them to expand a query, improving retrieval in its reported experiments. The risk is semantic drift: the language model can add an assumption that was not in the request, narrow an ambiguous query incorrectly, or introduce a hallucinated entity. A robust system usually retains the original query, records every transformation, and fuses or compares results rather than silently replacing the user's intent. Query rewriting should be evaluated on both recall gains and failure cases where the rewrite changes meaning. It is a retrieval tool, not permission to reinterpret the user's task. [9]

15. What is HyDE, and why must a hypothetical document generated by the model never be treated as evidence? [Advanced]

HyDE - Hypothetical Document Embeddings - asks a language model to generate a hypothetical document that would answer the query, embeds that generated text, and uses the embedding to retrieve nearby real documents. The generated document can contain false details; the HyDE paper explicitly acknowledges that it is 'fake.' Its role is to move the query into a representation that resembles relevant documents, not to supply factual evidence. The answer generator should still ground claims in retrieved corpus documents. This distinction is operationally important: if a system accidentally includes the hypothetical text as trusted context or cites it, the retrieval aid becomes a hallucinated source. HyDE is most interesting when labeled relevance data are unavailable and zero-shot dense retrieval is weak, but its benefits are workload-dependent and it adds an LLM call before retrieval. [10]

16. When can multi-query retrieval and rank fusion help, and why is generating more query variants not automatically better? [Advanced]

A single phrasing can miss relevant material, so a system may generate several query variants, retrieve for each, take a union, and fuse the rankings. Reciprocal Rank Fusion is useful here because it combines rank positions without requiring raw scores from different result lists to be calibrated. Multi-query retrieval can broaden recall when variants capture genuinely different vocabulary or facets. It can also increase latency, duplicate candidates, and amplify a bad rewrite across several searches. More variants therefore do not monotonically improve quality. Preserve the original query, deduplicate results, cap the expansion budget, and measure marginal recall from each added query. If the corpus is dominated by exact identifiers or the original query is already specific, extra variants may add mostly noise. The underlying principles - query expansion plus rank fusion - are more durable than any particular 'RAG-Fusion' recipe. [9][11]

17. Why can one-shot retrieval fail on multi-hop questions, and what does interleaving retrieval with reasoning change? [Advanced]

For a multi-hop question, the information needed for the second search may not be present in the original question. One retrieval call can therefore fail even if every required fact exists in the corpus. IRCoT demonstrated a method that interleaves retrieval with reasoning: earlier retrieved evidence and intermediate reasoning help formulate what to retrieve next, and new evidence then supports later reasoning steps. The architectural lesson is query decomposition and stateful retrieval, not that a chain-of-thought string itself must be exposed or trusted. Production systems can decompose the task into subquestions, retrieve for each, maintain explicit evidence state, and verify that the final answer is supported by the union of sources. Iterative retrieval costs more calls and can cascade an early mistaken decomposition, so multi-hop behavior should be invoked only when the task actually requires it and evaluated on hop-level evidence coverage. [12]

18. Should every request retrieve a fixed number of passages? How do active, adaptive, and corrective RAG approaches change that assumption? [Advanced]

Fixed 'retrieve top-k once for every request' is only one RAG policy. FLARE retrieves during long-form generation when upcoming content appears uncertain. Self-RAG trains a model to emit reflection signals that control when to retrieve and how to critique retrieved evidence and generations. CRAG adds a retrieval evaluator and chooses corrective actions when retrieved documents appear weak, including knowledge refinement and, in the paper's setup, external web search. These are different methods, but they share a principle: retrieval can be conditional and feedback-driven. The trade-off is complexity. A gating model can make its own mistakes, iterative retrieval adds latency, and web fallback changes the trust and freshness model. Use adaptive retrieval when the cost of retrieving everything is high or when tasks vary substantially in knowledge needs; do not assume a named method transfers unchanged to every system. [13][14][15]

19. What is context compression or selective augmentation, and when can compressing retrieved evidence make the system less reliable? [Advanced]

Retrieved passages can be much longer than the evidence actually needed. Context compression tries to keep high-value content while reducing tokens. RECOMP trains extractive or abstractive compressors and can output an empty string when retrieved documents add no useful information, combining compression with selective augmentation. LongLLMLingua compresses long prompts to reduce cost and address information density/position effects. The failure mode is obvious but serious: compression can delete a qualifier, number, negation, exception, or provenance cue that the answer depends on. Abstractive compression can also introduce generation error before the main generator runs. Treat compressed context as a derived representation; retain the original passages for verification and citations, and evaluate answer quality at several compression ratios. Compression is beneficial only if the saved tokens and latency outweigh preprocessing cost and lost evidence. [16][17]

20. How should a RAG system handle redundant, irrelevant, and mutually conflicting retrieved evidence? [Advanced - Reliability]

Redundancy, irrelevance, and conflict should be handled before the final prompt where possible. Deduplicate near-identical chunks so repeated copies do not consume context or create a false sense of corroboration. Rerank or filter irrelevant candidates rather than asking the generator to discover relevance inside a noisy prompt. For conflicting evidence, preserve source identity, timestamp, and provenance; do not silently merge contradictions into one statement. Research on conflicting retrieval shows that models can respond poorly to inconsistent evidence and may exhibit frequency or confirmation effects in particular experimental settings. A reliable application should surface material conflicts when they affect the answer, prefer authoritative or current sources according to explicit policy, and allow abstention when conflict cannot be resolved. 'Majority of retrieved chunks' is not a truth criterion because duplicate or poisoned content can create an artificial majority. [18][19]

21. What happens when retrieved evidence conflicts with the model's parametric knowledge? Which source will the model follow? [Advanced - Reliability]

There is no universal precedence rule. Depending on the model, prompt, topic familiarity, evidence quality, and conflict setup, a model may follow retrieved context, stick to its parametric prior, blend the two, or become

inconsistent. Studies of knowledge conflict have observed both over-reliance on external evidence and persistence of incorrect internal knowledge under different conditions. The engineering implication is to encode source policy explicitly and test it. If the application requires the provided corpus to be authoritative, say so, include provenance, and evaluate counterfactual cases where the model's memorized answer differs. If the corpus itself can be wrong, blind context obedience is also unsafe. For high-stakes conflicts, return the disagreement or seek another source rather than forcing the model to pick one. RAG changes the evidence available to the model; it does not automatically calibrate how the model arbitrates evidence. [18][19]

22. Why does attaching a citation to an answer not prove that the cited source supports the claim or that the model actually relied on it? [Advanced - Attribution]

A citation can fail in several different ways. The cited passage may not entail the claim; a claim may have no citation at all; the citation may support only part of a compound sentence; or the model may generate a correct-looking citation after producing an answer from parametric memory. ALCE evaluates cited generation and separates answer quality from citation quality. Later work distinguishes citation correctness - whether the source supports the claim - from citation faithfulness - whether the model genuinely relied on the cited evidence rather than post-rationalizing. In production, preserve a mapping from claims to source spans, check citation entailment or support, measure coverage of externally verifiable claims, and make source opening easy for users. A URL next to a sentence is metadata, not proof. Attribution is its own evaluation target. [20][21]

23. Why can a RAG system hallucinate even when the retriever returned the correct evidence? [Intermediate - Reliability]

Correct retrieval solves only the evidence-access problem. The generator can still overlook the passage, misread a table, make an arithmetic or reasoning error, combine evidence incorrectly, follow an irrelevant neighboring passage, or prefer its parametric prior. It can also generate unsupported details around an otherwise supported core answer. This is why RAG evaluation frameworks separate context relevance from answer faithfulness/relevance/correctness rather than treating retrieval recall as the final metric. A useful debugging test is an oracle-context experiment: give the generator only known supporting evidence. If the answer still fails, retrieval is no longer the main bottleneck. Conversely, if the oracle context works but normal RAG fails, inspect parsing, retrieval, reranking, or packing. RAG can reduce some factual errors, but it does not remove the need to evaluate the generator as a reasoning and evidence-use component. [22][23][24]

24. How should a production RAG system handle freshness, provenance, updates, deletions, and re-indexing? [Advanced - Production]

RAG is only as fresh as the source-to-index pipeline. Store stable source IDs, version or revision identifiers, timestamps, provenance, and ingestion status. Updates should propagate to all derived artifacts: parsed text, chunks, embeddings, lexical indexes, graph structures, caches, and citation mappings. Deletion is especially important because removing a source record but leaving an old embedding or cached prompt can continue to expose stale or prohibited text. Decide whether updates are event-driven, batched, or periodic based on the freshness SLO, and measure ingestion lag explicitly. During migrations, support dual indexes or versioned rollouts so queries do not mix incompatible representations. The original attraction of non-parametric memory was that knowledge can be updated without retraining the generator; that advantage only exists if the engineering system actually maintains index freshness and lineage. [1]

25. How should permissions be enforced in RAG so that unauthorized documents never become model context? [Advanced - Security]

Authorization must be a data-access boundary, not a natural-language instruction to the model. The system should determine the user's effective permissions and restrict candidate documents before unauthorized content is assembled into model context. Depending on the architecture, that can mean permission-aware filtering in the search layer, querying source systems under the caller's identity, or post-retrieval authorization before prompt construction - but never relying on the LLM to 'ignore' a document it was not allowed to see. Permissions must also propagate to cached results, summaries, graph nodes, and derived indexes. Recent enterprise access-control research argues for deterministic enforcement at both retrieval and generation boundaries because prompt sanitization and output filtering are probabilistic safeguards, not substitutes for authorization. Log which protected objects influenced each answer so access decisions are auditable. [30]

26. How can poisoned or instruction-bearing documents attack a RAG system, and why should retrieved text be treated as untrusted data? [Advanced - Security]

A RAG corpus expands the attack surface because retrieved text is external input. PoisonedRAG demonstrated that injecting malicious documents into a knowledge base can steer answers for targeted queries, and later security benchmarks show broader robustness problems with misleading or adversarial context. Even without an optimized poisoning attack, ordinary documents can contain instruction-like text that conflicts with system policy. Retrieved content should therefore be treated as untrusted data: preserve provenance and trust level, separate instructions from evidence in prompt design, restrict which sources can enter sensitive workflows, scan or validate ingestion where appropriate, enforce tool permissions outside the model, and monitor suspicious retrieval patterns. No

content filter is a proof of safety, so high-impact actions should have deterministic authorization and validation gates. The model should not gain new privileges merely because a retrieved document told it to. Indirect prompt-injection research specifically studies malicious instructions arriving through external content rather than directly from the user. [28][29][32]

27. Which metrics should you use to evaluate retrieval, context quality, generation, attribution, and the end-to-end RAG system? [Advanced - Evaluation]

Evaluate at multiple layers. For retrieval, use evidence-oriented measures such as Recall@k, Precision@k, MRR/nDCG where suitable, plus latency and coverage on real query types. For the assembled context, measure whether supporting evidence is present, how much irrelevant or duplicate material remains, and whether required permissions/provenance are correct. For generation, distinguish answer correctness from faithfulness to supplied evidence and from answer relevance. For attributed answers, measure citation support and citation completeness. End-to-end evaluation should also include abstention behavior, freshness, security cases, latency, cost, and task-specific business outcomes. A single aggregate score cannot tell you which module to fix. Build slice-level results for difficult categories such as multi-hop, exact identifiers, conflicting sources, stale documents, tables, and permission-filtered queries. RAGAS, ARES, and RAGChecker all reflect this multi-dimensional view. [22][23][24]

28. What do RAGAS, ARES, and RAGChecker actually evaluate, and why are automated RAG judges not ground truth? [Advanced - Evaluation]

RAGAS proposes reference-free metrics for dimensions such as context quality, answer faithfulness, and answer relevance using automated evaluation. ARES generates synthetic training data, trains lightweight judges for context relevance, answer faithfulness, and answer relevance, and uses a small human-annotated validation set with prediction-powered inference to improve aggregate estimates and confidence intervals. RAGChecker uses claim-level operations and a larger suite of retriever and generator diagnostic metrics, then meta-evaluates correlation with human judgments. They are useful tools, not interchangeable truth machines. LLM-based judges inherit model bias, prompt sensitivity, domain mismatch, and error. Synthetic questions can misrepresent production traffic. Use automated evaluation for iteration speed, but calibrate it against human-labeled examples from the real domain and periodically inspect disagreements. A metric is trustworthy only to the extent that its measurement behavior has been validated for your use case. [22][23][24]

29. What problem is GraphRAG designed to solve, and when is a graph-based index unnecessary or worse than ordinary RAG? [Advanced]

GraphRAG targets questions whose answer depends on relationships or global patterns across a corpus rather than one or two locally similar passages. Microsoft's GraphRAG builds an entity/relationship graph, clusters it into communities, and creates community summaries; global search can then reason over those summaries, while local modes use entity-centered context. The published work focuses particularly on corpus-level sensemaking questions where conventional local RAG is a poor fit. GraphRAG is not a universal upgrade. Indexing requires entity/relationship extraction and LLM-generated summaries, which adds cost, latency, storage, and new error modes. For direct fact lookup, exact identifiers, or small document collections, normal lexical/dense retrieval may be simpler and better. Use graph structure when the query genuinely needs relations, themes, or multi-document aggregation, and keep links back to original evidence because graph nodes and summaries are derived artifacts. [25]

30. Design a production RAG system for a changing enterprise corpus with permissions, citations, multi-hop questions, long documents, and a strict latency SLO. How would you prove each layer works? [Advanced - System Design]

Start with governance and workload, not a vector database. Ingestion should preserve source IDs, versions, structure, and ACLs; parsing must be tested on representative PDFs/tables. Build a simple lexical+dense baseline, measure evidence recall, then add reranking only if it improves the candidate-to-context step. Choose chunking from document structure and evaluation rather than a fixed token recipe; use hierarchical or contextualized retrieval only for failure cases they solve. Apply authorization before prompt assembly. For multi-hop questions, route to iterative retrieval; for corpus-level thematic questions, consider graph/hierarchical methods; for small bounded corpora, test long-context directly. Context packing should deduplicate, preserve provenance, order high-value evidence deliberately, and fit an explicit token budget. The generator should cite source spans and abstain when evidence is insufficient or conflicting. Security tests must include poisoned/instruction-bearing documents and permission boundaries. Prove the system with layer-specific evals plus end-to-end slices, latency/cost SLOs, freshness lag, and incident logging. Every added component needs a measured failure mode it fixes; otherwise it is architectural decoration. [3][25][26][27][30]

Depth checks

Area	What strong understanding looks like
Chunking	Explains the evidence-granularity trade-off and proposes corpus-specific evaluation instead of naming a magic token size.
Context packing	Treats top-k, deduplication, ordering, and compression as a token-budget optimization problem, not independent toggles.
Reliability	Separates retrieval failure from generation failure and understands that correct context can still yield an unsupported answer.
Attribution	Distinguishes answer correctness, support, citation coverage, and genuine reliance on evidence.
Security	Enforces authorization before context assembly and treats retrieved documents as untrusted input, not instructions.
Evaluation	Uses layer-specific metrics and representative slices, then validates automated judges against human-labeled production examples.

Additional concepts worth knowing

Prompt caching and context reuse. Repeated large context can sometimes be cached to reduce cost or latency, but cache identity and authorization become part of correctness.
Just-in-time agentic search. Agents can keep lightweight references and load information only when needed rather than preloading all potentially relevant context; this belongs to the broader context-engineering problem.
Structured and multimodal RAG. SQL, knowledge graphs, tables, images, and code often need retrieval or tool interfaces that preserve structure instead of flattening everything into text chunks.
Abstention and answerability. A production system should distinguish 'no relevant evidence found' from 'evidence found but insufficient/conflicting' and test whether the model declines appropriately.

References

[1] Patrick Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
[2] Anthropic. Effective context engineering for AI agents. Engineering blog, 2025.
[3] Yulong Hui, Yao Lu, Huanchen Zhang. UDA: A Benchmark Suite for Retrieval Augmented Generation in Real-world Document Analysis. NeurIPS 2024 Datasets and Benchmarks Track.
[4] Zhitong Wang et al. Document Segmentation Matters for Retrieval-Augmented Generation. Findings of ACL 2025.
[5] Anthropic. Introducing Contextual Retrieval. Engineering blog, 2024.
[6] Michael Gunther, Isabelle Mohr, Daniel James Williams, Bo Wang, Han Xiao. Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models. arXiv:2409.04701, 2024.
[7] Parth Sarthi et al. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR 2024.
[8] Nelson F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL 2023.
[9] Liang Wang, Nan Yang, Furu Wei. Query2doc: Query Expansion with Large Language Models. EMNLP 2023.
[10] Luyu Gao, Xueguang Ma, Jimmy Lin, Jamie Callan. Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). ACL 2023.
[11] Gordon V. Cormack, Charles L. A. Clarke, Stefan Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009.
[12] Harsh Trivedi et al. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. ACL 2023.
[13] Zhengbao Jiang et al. Active Retrieval Augmented Generation (FLARE). EMNLP 2023.
[14] Akari Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024.
[15] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, Zhen-Hua Ling. Corrective Retrieval Augmented Generation. arXiv:2401.15884, 2024.
[16] Fangyuan Xu, Weijia Shi, Eunsol Choi. RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation. ICLR 2024.
[17] Huiqiang Jiang et al. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. ACL 2024.
[18] Hung-Ting Chen, Michael Zhang, Eunsol Choi. Rich Knowledge Sources Bring Complex Knowledge Conflicts: Recalibrating Models to Reflect Conflicting Evidence. EMNLP 2022.
[19] Zhuoran Jin et al. Tug-of-War between Knowledge: Exploring and Resolving Knowledge Conflicts in Retrieval-Augmented Language Models. LREC-COLING 2024.
[20] Tianyu Gao, Howard Yen, Jiatong Yu, Danqi Chen. Enabling Large Language Models to Generate Text with Citations (ALCE). EMNLP 2023.
[21] Jonas Wallat, Maria Heuss, Maarten de Rijke, Avishek Anand. Correctness is not Faithfulness in RAG Attributions. ICTIR 2025.
[22] Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. EACL 2024 System Demonstrations.
[23] Jon Saad-Falcon, Omar Khattab, Christopher Potts, Matei Zaharia. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. NAACL 2024.
[24] Dongyu Ru et al. RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation. NeurIPS 2024 Datasets and Benchmarks Track.
[25] Darren Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, revised 2025.
[26] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, Michael Bendersky. Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach. EMNLP 2024 Industry Track.
[27] Kuan Li et al. LaRA: Benchmarking Retrieval-Augmented Generation and Long-Context LLMs - No Silver Bullet for LC or RAG Routing. ICML 2025.
[28] Wei Zou, Runpeng Geng, Binghui Wang, Jinyuan Jia. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. arXiv:2402.07867, 2024.
[29] Xun Liang et al. SafeRAG: Benchmarking Security in Retrieval-Augmented Generation of Large Language Model. ACL 2025.
[30] Shashank Shreedhar Bhatt et al. Enterprise AI Must Enforce Participant-Aware Access Control. arXiv:2509.14608, 2025.
[31] Valentin J. J. Kreileder, Johannes Reisinger, Andreas Fischer. Evaluating Chunking Strategies for Retrieval-Augmented Generation on Academic Texts. arXiv:2607.01852, 2026.
[32] Yulin Chen, Haoran Li, Yuan Sui, Yufei He, Yue Liu, Yangqiu Song, Bryan Hooi. Can Indirect Prompt Injection Attacks Be Detected and Removed? ACL 2025.